Pull Request Reviewer

A Pull Request Reviewer is an AI/software agent that reads a code change and gives review feedback like a senior engineer would — but in a structured, repeatable way. At its core, it answers - “What changed, what could break, what should improve, and what should the author do next?”

Functional Scope

1. To understand the intent of the change, the agent should infer the intent from branch names, commit messages, file names, and diff summary.
2. Review is performed locally from the IDE/ command palette using local Git state. GitHub/MCP integration is a future enhancement, not part of the current scope.
3. Find the difference of current branch (local) changes with the target branch locally. Another option is to read the Git PR directly. We choose to perform this locally.
4. We will only focus on reviewing the Java code. This can later be extended to review other file types like, SQL, Javascript etc.
5. Generate review report in .md file

Edge Cases

1. No Java files changed
2. Git working tree has uncommitted changes
3. Model returns invalid JSON
4. Duplicate findings
5. Network/model timeout

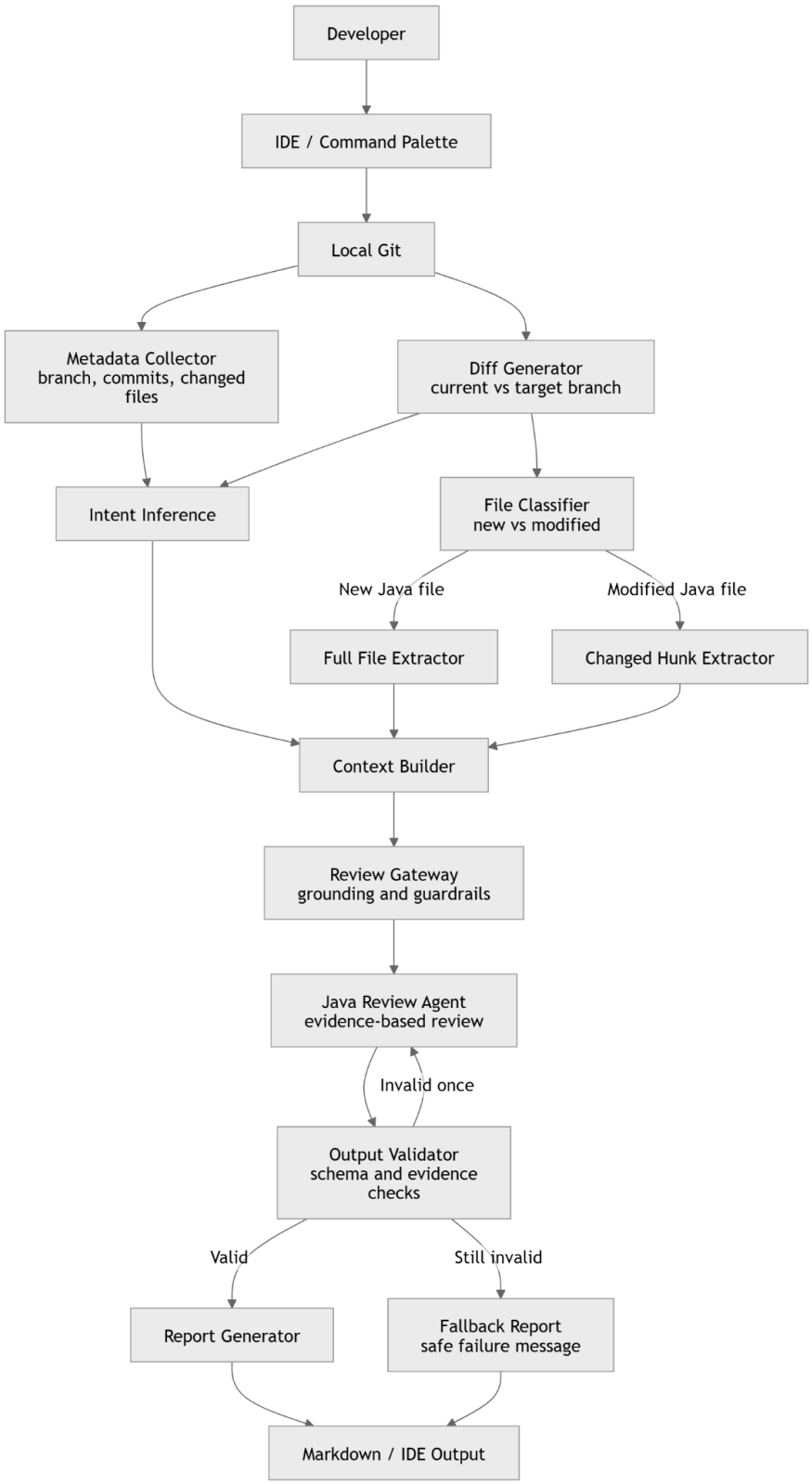
Non-functional requirements

1. Consistency > Availability
2. Reliability agent output must be reliable
3. Model timeout and retry

Core entities

User, IDE/ Command palette, Java file, Git, Output file

High Level Design



Trade-Offs

1. I chose local Git inspection instead of GitHub or GitLab APIs. That makes it easy to run from an IDE or terminal with no external dependency, but it means I have to infer review scope from branches, commits, and working tree state rather than a true pull request object.
2. I split the pipeline into LangGraph4j nodes like diff extraction, intent inference, context building, review, validation, and reporting. That adds some boilerplate and state-management overhead, but it makes the system easier to test, extend, and reason about.
3. The prompt is tuned to prefer correctness, security, testing, performance, maintainability, and only meaningful Java best practices. I explicitly avoid style-only or speculative comments. The trade-off is that the system may under-report softer suggestions, but the findings it does return are much more actionable.
4. Intent is inferred from branch names, commit messages, diff summary, file names, and optional developer notes. That's fast and cheap, but less accurate than doing a second model pass or deeper repository analysis.

If I have more time, I would

1. Integrate with GitHub/Git MCP to publish findings directly to pull requests instead of only generating local Markdown output.
2. Strengthen intent inference by integrating Jira for feature scope and acceptance criteria, and by adding a small intent-analysis model that looks at commit history, changed modules, and local repository context before the main code review pass.
3. SQL, JavaScript, Python review.

Sample output file generated after running the agent

1. Check ***ai-review.md*** uploaded in the Git repo

How I used AI tools

I used Codex to accelerate implementation, iteration, and refactoring. I made the design decisions, reviewed generated code, validated the workflow, and refined prompts, architecture, and trade-offs.